

DAY - 24: "Async JavaScript - async/await"

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#)  | [X: @_19_neeraj](#)  | [GitHub: Neeraj05042001](#)  |

Introduction

Async/await is a powerful feature in JavaScript that makes working with asynchronous code much easier and more readable. It's built on top of Promises and provides a cleaner, more synchronous-looking way to handle asynchronous operations without falling into callback hell.

Think of async/await as a way to write asynchronous code that reads like synchronous code, making it easier to understand and maintain.

What Will We Learn?

In this comprehensive guide, we'll cover:

- Understanding the need for async/await
- How to use the `async` and `await` keywords
- Error handling in async functions
- Top-level await
- Handling multiple async operations
- Real-world project implementation
- Best practices and common pitfalls

Why async/await?

Before async/await, we had to rely on callbacks and Promises, which could lead to complex nested code:

Promise-based approach:

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => {
    return fetch(`https://api.example.com/user/${data.userId}`);
  })
```

```
.then(response => response.json())
.then(user => {
  console.log(user);
})
.catch(error => console.error(error));
```

Async/await approach:

```
async function fetchUserData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    const userResponse = await fetch(`https://api.example.com/user/${data.userId}`);
    const user = await userResponse.json();
    console.log(user);
  } catch (error) {
    console.error(error);
  }
}
```

Knowledge Check

► Question: What are the main advantages of async/await over Promises?

The async Keyword

The `async` keyword is used before a function declaration to make it an asynchronous function. An async function always returns a Promise, even if you don't explicitly return one.

Basic Syntax:

```
async function myFunction() {
  return "Hello World";
}

// Arrow function syntax
const myAsyncFunction = async () => {
  return "Hello World";
};

// Method syntax
const obj = {
  async myMethod() {
    return "Hello World";
  }
}
```

```
}  
};
```

Key Points:

- Async functions always return a Promise
- If you return a value, it gets wrapped in a resolved Promise
- If you throw an error, it becomes a rejected Promise

Example:

```
async function foo() {  
    return Promise.resolve(101);  
}  
  
// This function returns a Promise that resolves to 101  
console.log(foo()); // Promise { <resolved>: 101 }
```

Knowledge Check

- Question: What happens when you return a regular value from an async function?

The await Keyword

The `await` keyword can only be used inside async functions. It pauses the execution of the function until the Promise is resolved or rejected.

Basic Syntax:

```
async function example() {  
    const result = await somePromise;  
    console.log(result);  
}
```

Important Rules:

1. `await` can only be used inside `async` functions
2. `await` pauses function execution until the Promise settles
3. If the Promise resolves, `await` returns the resolved value
4. If the Promise rejects, `await` throws the rejection reason

Example:

```
const promise = new Promise(function(resolve, reject) {
  setTimeout(() => {
    resolve("I am a promise");
  }, 1000)
});

async function handlePromise() {
  const result = await promise;
  console.log(result); // "I am a promise" (after 1 second)
}
```

Knowledge Check

- Question: Can you use await outside of an async function?

Is await Synchronous?

This is a common misconception. `await` is **not** synchronous - it's syntactic sugar that makes asynchronous code look synchronous.

Key Points:

- The function execution pauses at `await`, but the main thread continues
- Other code can run while waiting for the Promise to resolve
- The function resumes when the Promise settles

Example:

```
async function tacklePromise() {
  console.log("Before await");
  const result = await foo(); // Function pauses here
  console.log("After await:", result);
}

console.log("I am after tacklePromise()");
tacklePromise();
console.log("I continue executing");

// Output order:
// "I am after tacklePromise()"
// "Before await"
// "I continue executing"
// "After await: 101" (when Promise resolves)
```

Knowledge Check

► Question: Does await block the entire JavaScript thread?



Error Handling

Error handling in async/await is done using traditional `try/catch` blocks, which is much cleaner than `.catch()` with Promises.

Basic Error Handling:

```
async function handleErrors() {
  try {
    const result = await riskyOperation();
    console.log(result);
  } catch (error) {
    console.error("An error occurred:", error);
  }
}
```

Complete Example:

```
const errorPromise = new Promise((resolve, reject) => {
  reject("Error Occured!");
});

async function handleErrorPromise() {
  try {
    const result = await errorPromise;
    console.log(result);
  } catch(error) {
    console.error("Caught error:", error);
  }
}

handleErrorPromise(); // Outputs: "Caught error: Error Occured!"
```

Advanced Error Handling:

```
async function robustErrorHandling() {
  try {
    const data = await fetchData();
    const processedData = await processData(data);
    return processedData;
  } catch (error) {
    if (error.name === 'NetworkError') {
      console.log('Network issue, retrying...');
    }
  }
}
```

```
// Retry logic
} else if (error.name === 'ValidationError') {
  console.log('Data validation failed');
  // Handle validation error
} else {
  console.log('Unexpected error:', error);
  throw error; // Re-throw if we can't handle it
}
} finally {
  console.log('Cleanup operations');
}
}
```



Knowledge Check

- Question: How do you handle errors in async/await functions?



Project - Weather App Concepts

When building a weather app with async/await, you would typically:

1. Fetch weather data from an API
2. Handle loading states
3. Manage errors gracefully
4. Update the UI based on results

```
async function getWeatherData(city) {
  try {
    // Show loading state
    showLoading(true);

    const response = await fetch(`https://api.weather.com/v1/current?city=${city}`);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const weatherData = await response.json();
    displayWeather(weatherData);

  } catch (error) {
    displayError(`Failed to fetch weather: ${error.message}`);
  } finally {
    showLoading(false);
  }
}
```



Top Level await

Top-level await allows you to use `await` outside of async functions in ES modules. This is useful for module initialization.

Traditional Approach (IIFE):

```
let stores;
(async () => {
  let response = await fetch('http://localhost:3000/api/pizzahub');
  stores = await response.json();
  console.log(stores);
})(); // Immediately Invoked Function Expression

console.log(stores); // undefined (executes before async function completes)
```

With Top-Level Await (ES2022):

```
// In an ES module
const response = await fetch('http://localhost:3000/api/pizzahub');
const stores = await response.json();
console.log(stores); // Works correctly
```

Important Notes:

- Only works in ES modules (type="module")
- Can delay module loading
- Use sparingly for initialization tasks



Knowledge Check

- Question: When can you use top-level await?



Handling Multiple async/await

When dealing with multiple asynchronous operations, you have several strategies:

Sequential Execution:

```
async function sequential() {
  const result1 = await operation1(); // Waits for completion
```

```
const result2 = await operation2(); // Waits for result1 to complete
const result3 = await operation3(); // Waits for result2 to complete
return [result1, result2, result3];
}
```

Parallel Execution:

```
async function parallel() {
  // Start all operations simultaneously
  const promise1 = operation1();
  const promise2 = operation2();
  const promise3 = operation3();

  // Wait for all to complete
  const results = await Promise.all([promise1, promise2, promise3]);
  return results;
}
```

Using Promise.allSettled():

```
async function handleMultipleWithSettled() {
  const results = await Promise.allSettled([
    fetch(URL1).then(r => r.json()),
    fetch(URL2).then(r => r.json()),
    fetch(URL3).then(r => r.json())
  ]);

  results.forEach((result, index) => {
    if (result.status === 'fulfilled') {
      console.log(`Request ${index + 1} succeeded:`, result.value);
    } else {
      console.log(`Request ${index + 1} failed:`, result.reason);
    }
  });
}
```

Real Example (Pokemon API):

```
const BULBASAUR_URL = 'https://pokeapi.co/api/v2/pokemon/bulbasaur';
const RATICATE_URL = 'https://pokeapi.co/api/v2/pokemon/raticate';
const KAKUNA_URL = 'https://pokeapi.co/api/v2/pokemon/kakuna';

async function resolvePokemonsV2() {
  const responses = await Promise.allSettled([
    fetch(BULBASAUR_URL).then(response => response.json()),
    fetch(RATICATE_URL).then(response => response.json()),
    fetch(KAKUNA_URL).then(response => response.json())
  ]);
}
```



```
]);

console.log(responses);
// Each response contains: { status: 'fulfilled'/'rejected', value/reason: ... }
}
```

Knowledge Check

- Question: What's the difference between `Promise.all()` and `Promise.allSettled()`?

The Pizzahub with `async/await`

Converting Promise-based code to `async/await` makes it more readable:

Before (Promises):

```
fetch('http://localhost:3000/api/pizzahub')
  .then(response => response.json())
  .then(stores => {
    console.log(stores);
    return fetch(`http://localhost:3000/api/pizzahub/${stores[0].id}/menu`);
  })
  .then(response => response.json())
  .then(menu => console.log(menu))
  .catch(error => console.error(error));
```

After (`async/await`):

```
async function getPizzaData() {
  try {
    const storeResponse = await fetch('http://localhost:3000/api/pizzahub');
    const stores = await storeResponse.json();
    console.log(stores);

    const menuResponse = await fetch(`http://localhost:3000/api/pizzahub/${stores[0].id}/menu`);
    const menu = await menuResponse.json();
    console.log(menu);
  } catch (error) {
    console.error('Error fetching pizza data:', error);
  }
}
```



Common Patterns and Best Practices

1. Always Handle Errors

```
async function goodPractice() {
  try {
    const result = await riskyOperation();
    return result;
  } catch (error) {
    console.error('Operation failed:', error);
    throw error; // Re-throw if the caller needs to handle it
  }
}
```

2. Avoid Mixing async/await with .then()

```
// ❌ Don't do this
async function mixed() {
  const result = await somePromise().then(data => data.value);
  return result;
}

// ✅ Do this instead
async function clean() {
  const data = await somePromise();
  return data.value;
}
```

3. Use Promise.all() for Independent Operations

```
// ❌ Sequential (slower)
async function sequential() {
  const user = await fetchUser();
  const posts = await fetchPosts();
  const comments = await fetchComments();
  return { user, posts, comments };
}

// ✅ Parallel (faster)
async function parallel() {
  const [user, posts, comments] = await Promise.all([
    fetchUser(),
    fetchPosts(),
    fetchComments()
  ]);
}
```

```
return { user, posts, comments };  
}
```

Common Interview Questions

- ▶ Q1: What is the difference between async/await and Promises?
 - ▶ Q2: Can you use await without async?
 - ▶ Q3: What happens if you don't await an async function?
 - ▶ Q4: How do you handle multiple async operations that don't depend on each other?
 - ▶ Q5: What is the purpose of Promise.allSettled() vs Promise.all()?
-

Practice Tasks

Basic Level (1-3)

1. Convert Promise to async/await

```
// Convert this Promise-based code to use async/await  
function fetchUserData() {  
  return fetch('https://jsonplaceholder.typicode.com/users/1')  
    .then(response => response.json())  
    .then(user => {  
      console.log(user);  
      return user;  
    })  
    .catch(error => {  
      console.error('Error:', error);  
      throw error;  
    });  
}
```

▶ Answer

2. Error Handling Create an async function that handles different types of errors when fetching data from an API.

▶ Answer

3. Sequential vs Parallel Write two functions: one that fetches three pieces of data sequentially, and another that fetches them in parallel.

► Answer

Intermediate Level (4-6)

4. Retry Logic Create an async function that retries a failed operation up to 3 times with exponential backoff.

► Answer

5. Timeout Wrapper Create a function that adds timeout functionality to any async operation.

► Answer

6. Batch Processing Create a function that processes an array of items in batches using async/await.

► Answer

Advanced Level (7-10)

7. Rate Limiting Implement a rate-limited async function that ensures no more than N operations execute per second.

► Answer

8. Async Queue Implement an async queue that processes items one at a time in order.

► Answer

9. Async Pipeline Create a pipeline that passes the result of one async operation to the next.

► Answer

10. Advanced Error Recovery Create a robust async function that implements circuit breaker pattern for external API calls.

► Answer

Summary

Async/await is a powerful feature that makes asynchronous JavaScript code more readable and maintainable. Key takeaways:

Essential Concepts:

- **async** functions always return Promises
- **await** pauses function execution until Promise settles

- Use **try/catch** for error handling
- **await** can only be used inside async functions (except top-level await)

Best Practices:

- Always handle errors with try/catch
- Use Promise.all() for parallel operations
- Avoid mixing async/await with .then()
- Consider using Promise.allSettled() when you need all results regardless of failures
- Implement proper error recovery strategies for production code

Performance Considerations:

- Sequential await calls are slower than parallel execution
- Use Promise.all() when operations don't depend on each other
- Consider implementing rate limiting and circuit breakers for external APIs
- Be mindful of memory usage with large datasets

Common Pitfalls:

- Forgetting to await async functions
- Using await in loops without considering performance
- Not handling errors properly
- Mixing async/await with Promise chains unnecessarily

Master these concepts and you'll be able to write clean, efficient asynchronous JavaScript code that's easy to read, debug, and maintain!

Next Steps

Tomorrow, we'll likely cover:

- **Fetch API in depth**
- **Working with REST APIs**
- **Handling different HTTP methods**
- **Request and response interceptors**
- **Building a complete API client**

Keep practicing async/await patterns, as they're fundamental to modern JavaScript development!



Let's Connect & Level Up Together

Follow me for daily JavaScript tips, insightful notes, and project-based learning.



X (Twitter)



LinkedIn



Telegram



Threads



Dive into the full notes on GitHub → **40 Days JavaScript**

© 2025 • Crafted with ❤️ by Neeraj



Next: Day - 25: `fetch()`



Happy JavaScript coding!

